

ARIN7013 Exam Cheat Sheet, Page 1: CNN, Kernels, Linear Systems

Use this as a hand-copy plan: formula first, then method steps, then final conclusion.

CNN And Convolution

Cross-correlation. For image I and filter K ,

$$(I * K)_{ij} = \sum_{a,b} I_{i+a,j+b} K_{ab}.$$

In CNN slides, “convolution” usually means cross-correlation: no filter flip.

Output shape. For input $H \times W \times C_{\text{in}}$, filter $F_h \times F_w$, padding P_h, P_w , stride S_h, S_w ,

$$H_{\text{out}} = \left\lfloor \frac{H + 2P_h - F_h}{S_h} \right\rfloor + 1, \quad W_{\text{out}} = \left\lfloor \frac{W + 2P_w - F_w}{S_w} \right\rfloor + 1.$$

Output channels = C_{out} , the number of filters.

Weights.

$$\# \text{conv weights} = F_h F_w C_{\text{in}} C_{\text{out}}$$

Ignore bias if question says so; otherwise add C_{out} . Pooling has no trainable weights. Flatten only re-shapes. Fully connected: input dimension $\times$ output dimension.

CNN advantages. Write: local receptive fields, sparse connectivity, parameter sharing, fewer parameters, grid/image inductive bias, translation-related robustness after convolution/pooling.

Residual CNN. Plain block learns $F(X)$. Residual block:

$$Y = X + F(X), \quad F(X) = K_2 * \sigma(K_1 * X).$$

Need same shape for addition. If shape/channel differs, use projection, e.g. 1×1 conv. Advantage: easier optimization, learns correction, better gradient flow.

Kernels And RKHS

Positive definite kernel. $K : X \times X \rightarrow \mathbb{R}$ is p.d. if for any x_i and c_i ,

$$\sum_{i,j} c_i c_j K(x_i, x_j) \geq 0.$$

Equivalent Gram matrix $[K(x_i, x_j)]$ is positive semidefinite.

Inner product proof. If $K(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{H}}$, then

$$\sum_{i,j} c_i c_j K(x_i, x_j) = \left\| \sum_i c_i \phi(x_i) \right\|_{\mathcal{H}}^2 \geq 0.$$

Closure rules. If K_1, K_2 are p.d., then $aK_1 + bK_2$ for $a, b \geq 0$, $K_1 K_2$, pointwise limits, and $\exp(K_1)$ are p.d. Polynomial $(1 + \langle x, x' \rangle)^m$ is p.d.

Gaussian kernel proof.

$$e^{-\gamma \|x - x'\|^2} = e^{-\gamma \|x\|^2} e^{2\gamma x^T x'} e^{-\gamma \|x'\|^2}.$$

Use

$$e^{2\gamma x^T x'} = \sum_{m=0}^{\infty} \frac{(2\gamma)^m}{m!} (x^T x')^m,$$

nonnegative sum of p.d. kernels, then multiply by $g(x)g(x')$.

Aronszajn theorem. K is p.d. iff there exists a Hilbert space $\mathcal{H}$ and feature map ϕ such that

$$K(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{H}}.$$

Exam sentence: kernels allow inner products in feature space without explicitly constructing features.

RKHS center distance. For $\bar{\phi} = \frac{1}{n} \sum_j \phi(x_j)$,

$$\|\phi(x_i) - \bar{\phi}\|_{\mathcal{H}}^2 = K_{ii} - \frac{2}{n} \sum_j K_{ij} \frac{1}{n^2} \sum_{j,l} K_{jl}.$$

Average distance: average square root. Average squared distance: average this quantity.

KRR

Objective in slides:

$$\hat{f} = \arg \min_{f \in \mathcal{H}} \frac{1}{n} \sum_i |f(x_i) - y_i|^2 + \beta \|f\|_{\mathcal{H}}^2.$$

Representer theorem:

$$\hat{f}(x) = \sum_i \varsigma_i K(x_i, x).$$

Let Gram $K_{ij} = K(x_i, x_j)$, $y = (y_i)$. Then fitted values $K\varsigma$ and $\|\hat{f}\|_{\mathcal{H}}^2 = \varsigma^T K \varsigma$.

$$\varsigma = (K + \beta n I)^{-1} y$$

up to null-space ambiguity; this choice is the standard answer. Final predictor must be written as the kernel expansion, not just the coefficient vector.

Linear Systems

Gaussian elimination / LU. Direct method; cost about $O(n^3)$, solve triangular systems in $O(n^2)$. LU solve template:

$$A = LU, \quad Ly = b, \quad Ux = y.$$

Never write $x = A^{-1}b$ as a computational method unless asked conceptually.

Doolittle LU. $A = LU$, $L_{kk} = 1$.

$$u_{kj} = a_{kj} - \sum_{s=1}^{k-1} \ell_{ks} u_{sj}, \quad j = k, \dots, n,$$

$$\ell_{ik} = \frac{a_{ik} - \sum_{s=1}^{k-1} \ell_{is} u_{sk}}{u_{kk}}, \quad i = k+1, \dots, n.$$

Need nonzero pivot u_{kk} . Pivoting swaps rows when pivot is zero/small; partial pivoting uses largest available pivot in column.

2025 tridiagonal LU pattern. For A_n with diagonal 2 and off-diagonal -1 :

$$U_{ii} = \frac{i+1}{i}, \quad U_{i,i+1} = -1, \quad L_{ii} = 1, \quad L_{i+1,i} = -\frac{i}{i+1}.$$

Derive by small cases $n = 3, 4, 5$: $d_1 = 2$, $\ell_{i+1,i} = -1/d_i$, $d_{i+1} = 2 - 1/d_i$.

Condition number.

$$\kappa(A) = \|A\| \|A^{-1}\|.$$

Large $\kappa(A)$ means small perturbations in b or A can cause large solution error. Ill-conditioned $\approx$ nearly singular.

Jacobi / Gauss-Seidel / SVD

Jacobi. Write $A = D + R$:

$$x^{(k+1)} = D^{-1}(b - Rx^{(k)}), \quad x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right).$$

All RHS values are old values. Easy to parallelize. Iteration matrix:

$$G_J = -D^{-1}R = I - D^{-1}A, \quad e^{(k+1)} = G_J e^{(k)}.$$

Converges for all $x^{(0)}$ and b iff $\rho(G_J) < 1$. Sufficient: strict diagonal dominance.

Gauss-Seidel. Uses newest values immediately:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right).$$

Usually faster if convergent, less parallel.

Matrix multiplication. $(n \times n)(n \times n)$: $O(n^3)$. $(m \times n)(n \times r)$: $O(mnr)$.

SVD.

$$A = U \Sigma V^T, \quad A^T A v_i = \sigma_i^2 v_i, \quad u_i = \frac{A v_i}{\sigma_i}.$$

Rank- k approximation:

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T.$$

Best in Frobenius/spectral norm:

$$\|A - A_k\|_F^2 = \sum_{i > k} \sigma_i^2, \quad \|A - A_k\|_2 = \sigma_{k+1}.$$

ARIN7013 Exam Cheat Sheet, Page 2: Eigen, PCA, ODE, ResNet, Neural ODE

Most marks come from executing the algorithm with the lecture notation.

Eigenvalue Algorithms

Rayleigh quotient. For symmetric A ,

$$R_A(v) = \frac{v^T A v}{v^T v}, \quad \text{if } \|v\| = 1, \quad R_A(v) = v^T A v.$$

Eigenvectors are stationary points. Largest eigenvalue maximizes $R_A(v)$ over unit vectors.

Power iteration. For dominant eigenvalue $|\lambda_1| > |\lambda_2|$ and $x_0 = a_1 u_1 + \dots$, $a_1 \neq 0$:

$$z^{(k)} = A v^{(k-1)}, \quad v^{(k)} = \frac{z^{(k)}}{\|z^{(k)}\|}, \quad \omega^{(k)} = (v^{(k)})^T A v^{(k)}.$$

Proof template:

$$A^k x_0 = a_1 \lambda_1^k \left(u_1 + \sum_{i \geq 2} \frac{a_i}{a_1} \left(\frac{\lambda_i}{\lambda_1} \right)^k u_i \right).$$

Convergence factor roughly $|\lambda_2/\lambda_1|$.

Inverse iteration. Choose shift ν near desired eigenvalue:

$$(A - \nu I) z^{(k)} = v^{(k-1)}, \quad v^{(k)} = \frac{z^{(k)}}{\|z^{(k)}\|}.$$

Targets eigenvalue closest to ν because eigenvalues of $(A - \nu I)^{-1}$ are $1/(\lambda_i - \nu)$.

Rayleigh quotient iteration.

$$\omega^{(k-1)} = (v^{(k-1)})^T A v^{(k-1)},$$

$$(A - \omega^{(k-1)} I) z^{(k)} = v^{(k-1)}, \quad v^{(k)} = \frac{z^{(k)}}{\|z^{(k)}\|}.$$

For symmetric A , sufficiently close to simple eigenpair: cubic convergence.

Ordinary PCA

Core notation. Data $x_i \in \mathbb{R}^d$. Center unless question says not to:

$$\tilde{x}_i = x_i - m, \quad m = \frac{1}{n} \sum_i x_i.$$

Covariance:

$$V = \frac{1}{n} \sum_i \tilde{x}_i \tilde{x}_i^T = \frac{1}{n} X^T X$$

if rows are samples. If columns are samples, use $V = \frac{1}{n} X X^T$.

Loading vs score. Principal component asked in exams usually means loading vector:

$$V h_j = \omega_j h_j, \quad \|h_j\| = 1.$$

PC score for datum x :

$$y_j = \langle x, h_j \rangle \quad \text{or} \quad y_j = \langle \tilde{x}, h_j \rangle.$$

Do not answer with score when asked for loading/component.

Projection and MSE. Projection onto first s PCs:

$$P_s(x_i) = \sum_{j=1}^s \langle x_i, h_j \rangle h_j.$$

Average reconstruction MSE:

$$\frac{1}{n} \sum_i \|x_i - P_s(x_i)\|^2 = \sum_{j>s} \omega_j.$$

Information preserved:

$$\frac{\sum_{j=1}^s \omega_j}{\sum_{j=1}^d \omega_j} \times 100\%.$$

If d is large. Use XX^T/n or $X^T X/n$ trick. Nonzero eigenvalues match up to dimension convention. For SVD $X = U \Sigma V^T$, PCA directions are singular vectors; variances are σ_i^2/n .

Kernel PCA

Setup. Feature map $\Psi : X \rightarrow \mathcal{H}$, kernel

$$K(x, x') = \langle \Psi(x), \Psi(x') \rangle_{\mathcal{H}}, \quad K_{ij} = K(x_i, x_j).$$

Assume centered in feature space unless question says no centering. If no centering, use raw Gram matrix.

Eigenproblem. Slide covariance:

$$V = \frac{1}{n} \sum_i \Psi(x_i) \Psi(x_i)^T, \quad v_j = \sum_i a_{ji} \Psi(x_i).$$

Here $v_j \in \mathcal{H}$ is the loading/eigenfunction. It is not an n -vector. Then

$$K a_j = n \omega_j a_j, \quad a_j^T K a_j = 1.$$

Exam calculation route:

$$K q_j = \rho_j q_j, \quad \|q_j\| = 1, \quad a_j = \frac{q_j}{\sqrt{\rho_j}}, \quad \rho_j = n \omega_j.$$

Then actual RKHS PC:

$$v_j = \sum_i a_{ji} \Psi(x_i).$$

$q_j, a_j \in \mathbb{R}^n$ are coefficient vectors only.

Projection and MSE. For new x :

$$y_j(x) = \langle \Psi(x), v_j \rangle_{\mathcal{H}} = \sum_i a_{ji} K(x_i, x).$$

Training score vector = $K a_j$, not the loading. Using first s kernel PCs:

$$\text{MSE} = \frac{1}{n} \left(\text{tr}(K) - \sum_{j=1}^s \rho_j \right) = \sum_{j>s} \omega_j.$$

Common trap: answer must be $v_j \in \mathcal{H}$, represented by a_j , not score vector $K a_j$.

ODE Solvers

IVP.

$$x'(t) = f(t, x), \quad x(t_0) = x_0, \quad t_i = t_0 + i h.$$

Theta method.

$$x_{i+1} = x_i + h[(1 - \theta)f(t_i, x_i) + \theta f(t_{i+1}, x_{i+1})].$$

$\theta = 0$: Forward Euler, explicit.

$$x_{i+1} = x_i + h f(t_i, x_i).$$

$\theta = 1$: Backward Euler, implicit.

$$x_{i+1} = x_i + h f(t_{i+1}, x_{i+1}).$$

$\theta = 1/2$: trapezoidal, generally implicit,

$$x_{i+1} = x_i + \frac{h}{2} [f(t_i, x_i) + f(t_{i+1}, x_{i+1})].$$

2024 explicit improved Euler scheme.

$$x_i = x_{i-1} + \frac{h}{2} \{ f(t_{i-1}, x_{i-1}) + f(t_i, x_{i-1} + h f(t_{i-1}, x_{i-1})) \}.$$

It is explicit: RHS uses known x_{i-1} only.

Error estimate template. Define exact/numerical error

$$e_i = x(t_i) - x_i.$$

Local truncation/consistency error, example theta 1/2:

$$T_i = \frac{x(t_{i+1}) - x(t_i)}{h} - \frac{1}{2} [f(t_i, x(t_i)) + f(t_{i+1}, x(t_{i+1}))].$$

Taylor with bounded derivatives gives $|T_i| \leq Ch^2$ for trapezoidal/improved Euler, and $|T_i| \leq Ch$ for Forward Euler.

Subtract numerical scheme from exact consistency relation. Use Lipschitz:

$$|f(t, x) - f(t, y)| \leq L|x - y|.$$

Then get recurrence like

$$|e_{i+1}| \leq (1 + Ch)|e_i| + Ch^{p+1}.$$

Discrete Gronwall:

$$\max_i |e_i| \leq Ch^p.$$

Forward Euler global order 1; trapezoidal/improved Euler global order 2.

ResNet And Neural ODE

ResNet as Euler. Forward Euler for hidden state:

$$u_{l+1} = u_l + h f(u_l, \theta_l).$$

ResNet block:

$$u_{l+1} = u_l + F(u_l, \theta_l).$$

Interpret depth as time steps; residual connection improves gradient flow and stability.

Neural ODE.

$$\frac{dx(t)}{dt} = f(x(t), t, \theta), \quad x(t_1) = \text{ODESolve}(f, x(t_0), t_0, t_1)$$

Continuous-depth model; solver chooses steps adaptively.

Adjoint gradient. Loss $L = L(x(t_1))$. Define adjoint

$$a(t) = \frac{\partial L}{\partial x(t)}.$$

Terminal condition:

$$a(t_1) = \frac{\partial L}{\partial x(t_1)}.$$

Backward adjoint ODE:

$$\frac{da(t)}{dt} = -a(t)^T \frac{\partial f(x(t), t, \theta)}{\partial x}.$$

Parameter gradient:

$$\frac{\partial L}{\partial \theta} = \int_{t_0}^{t_1} a(t)^T \frac{\partial f(x(t), t, \theta)}{\partial \theta} dt.$$

Memory comparison: ResNet/backprop stores many layer activations; Neural ODE adjoint can recompute trajectory backward, lower memory but possible numerical instability/recomputation cost.

Last-Minute Traps

- PCA component/loading $\neq$ PC score. Loading is eigenvector.
- Kernel PCA: answer component is $v \in \mathcal{H}$; normalize $a^T K a = 1$.
- KRR: write final function $\hat{f}(x) = \sum_i \varsigma_i K(x_i, x)$.
- Jacobi uses all old values; Gauss-Seidel uses new values immediately.
- CNN: shape formula before weight count; filter weights are shared.
- Do not center data if exam explicitly says no centering.
- ODE error proof: truncation error $\rightarrow$ recurrence $\rightarrow$ Gronwall/order.
- Residual addition requires matching dimensions.